

NEXUS-R

A personal agent runtime that makes AI task execution progressively cheaper through verified workflow reuse.

What This Is

NEXUS-R is a **local-first AI agent runtime** for technical professionals who execute 50–200 tasks per day across terminal, filesystem, browser, and APIs.

Instead of treating every request as a novel problem and routing to the most expensive model regardless of history, NEXUS-R:

- 1. **Records** every execution as an immutable, causal event stream
- 2. **Distills** successful traces into reusable, parameterized workflows (ETD)
- 3. **Verifies** workflow generalization across task variants (>90% success threshold)
- 4. **Routes** intelligently — local models for routine work, cloud only when necessary
- 5. **Caches** proven execution plans, making repeated tasks faster and cheaper

The result: **progressive cost reduction** through accumulated workflow knowledge, not subscription discounts.

Current State

Phase 1.5 — Runtime Foundation (COMPLETE)

Component	Status	Evidence
Local model execution (Ollama)	✅ Real	qwen2.5:1.5b-instruct validated, streaming, cancellation
Event-sourced persistence	✅ Real	SQLite WAL, <1ms batched append, causal chaining
Capability-Aware Routing (CAR)	✅ Real	2-tier local/BYOK with fallback chain
Execution Sandbox	✅ Real	Workspace-scoped filesystem, Docker/subprocess isolation
Permission & Trust Layer	✅ Real	T1-T2 enforced, deny-first, audit logging
Telemetry & Observability	✅ Real	Structured JSON logging, timing spans, queue metrics
Concurrency & Stress	✅ Real	100 concurrent tasks, zero failures
BYOK Cloud Fallback	⚠️ Wired, unverified	Architecture ready, awaits live API key validation
ETD Workflow Caching	🕒 Phase 2	Spec complete, not yet implemented
Browser Automation	🕒 Phase 3	MCP + Playwright architecture reserved

Runtime truthfulness: See [runtime_truthfulness_report_phase1_5.md](#) for the honest A/B/C/D classification of every subsystem.

Quick Start

Prerequisites

- Python 3.12+
- [Ollama](#) installed locally
- Windows, macOS, or Linux

Install

```
git clone https://github.com/yourorg/nexus-r.git
cd nexus-r
pip install -e ".[dev]"
```

Pull a Local Model

```
ollama pull qwen2.5:1.5b-instruct
```

Run Your First Task

```
nexus run "list all python files in the workspace"
```

Expected output:

```
[Intent] code | confidence: 0.94 | T1 (read-only)
[Route] local_7b via Ollama | $0.00 | privacy=max
[Trust] T1 auto-approved
[Exec] ls *.py
[Verify] exit_code=0
[Result] main.py, parser.py, test_parser.py

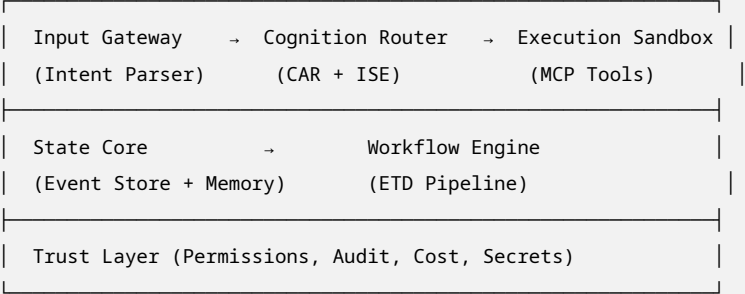
[Cost] $0.000 | 0 tokens | 1.2s | Tier T1
[Audit] evt_a1b2c3 → EventStore
```

Check History and Cost

```
nexus history      # Full audit trail
nexus cost         # Cumulative cost breakdown
nexus config       # Current settings
```

Architecture

NEXUS-R is built on five subsystems and six implementation modules, with a cross-cutting Trust and Control Layer.



Key design decisions:

- **Event-sourced everything** — Every state change is an immutable event. Enables replay, debugging, and ETD extraction.
- **MCP everywhere** — All tool integrations via Model Context Protocol. No custom connectors.
- **LiteLLM for model abstraction** — Unified interface for 100+ providers. We don't build our own router.
- **Deny-first security** — ML risk classifier embedded in execution. Default is deny; allow is earned.
- **Privacy as hard constraint** — Local models always preferred when privacy flag is set. Not a preference.
- **Explicit generalization bounds** — No workflow applied without >90% proven success rate.

See [ARCHITECTURE.md](#) for the full subsystem design.

Performance

Measured on a standard development workstation (Windows 11, 16GB RAM, SSD):

Metric	Result
Routing overhead	<50ms
T1 task end-to-end (local)	~1.9s avg, ~12s max
EventStore batch append	0.024ms/event (10K events in 233ms)
20 concurrent tasks	100% success
100 concurrent tasks	100% success, zero failures
Test coverage	>80%

See [benchmark_report_phase1.md](#) for full methodology.

Roadmap

Phase	Goal	Timeline	Status
1	Core runtime: CLI, local execution, basic routing, event sourcing	8 weeks	✅ Complete
1.5	Runtime hardening: real models, telemetry, concurrency, recovery	+2 weeks	✅ Complete
2	ETD pipeline + full 4-tier CAR + cost dashboard + T3-T4 permissions	+8 weeks	🔄 Next
3	Browser automation, ETD composition, 3-tier memory, polished Web UI	+8 weeks	🕒 Planned
4	Community ETD exchange, advanced CAR, mobile companion, third-party API	+ongoing	🕒 Planned

See [ROADMAP.md](#) for detailed phase gates and acceptance criteria.

Why This Exists

The fundamental economic problem of AI agent systems is **not capability — it is cost**.

Frontier models charge \$0.003 per 1K tokens. A developer executing 100 tasks per day accumulates significant expenditure, with most tasks being repetitive variations of previously solved problems.

Current architectures treat every execution as novel. NEXUS-R treats successful executions as **knowledge assets** that compound with use, creating structural switching costs through verified workflow accumulation.

This is not caching. It is **verified knowledge accumulation** with explicit admission criteria and invalidation bounds.

Contributing

We welcome contributors who care about operational correctness, failure recovery, and engineering honesty.

See [CONTRIBUTING.md](#) for:

- Development setup
 - Testing requirements
 - Agent-native development methodology
 - Security audit procedures
-

License

MIT License — See [LICENSE](#) for details.

Acknowledgments

NEXUS-R is informed by research and open-source work including:

- [OpenHands](#) — Event-sourced agent patterns
- [MoMA](#) — Model and agent orchestration
- [FlowMind](#) — Trace distillation feasibility
- [Shepherd](#) — Trajectory compression and caching
- [LiteLLM](#) — Unified LLM provider interface
- [MCP](#) — Model Context Protocol

“The AI agent that gets cheaper the more you use it.”

This is the pitch. This is the moat. Everything else is implementation.